As I discussed in section 27.1, IIS acts as a reverse proxy for your ASP.NET Core app. That means IIS needs to be able to communicate directly with your app to forward incoming requests to and outgoing responses from your app.

IIS handles this with the ASP.NET Core Module, but a certain degree of negotiation is required between IIS and your app. For this to work correctly, you need to configure your app to use IIS integration.

IIS integration is added automatically when you use `WebApplicationBuilder`, so there's typically nothing more to do. However, in chapter 30 you'll learn about the generic host and how to create custom application builders using `HostBuilder`. If your app uses a customer application builder and you want to use IIS, you need to ensure that you add IIS integration with the `UseIIS()` or `UseIISIntegration()` extension methods:

- `UseIIS()` configures your application to support IIS with an in-process hosting model.
- `UseIISIntegration()` configures your application to support IIS with an out-of-process hosting model.

These methods are automatically called by `WebApplicationBuilder`, but if you're not using your application with IIS, the `UseIIS()` and `UseIISIntegration()` methods will have no effect on your app, so it's safe to include them anyway.

In-process vs. out-of-process hosting in IIS